Kafka

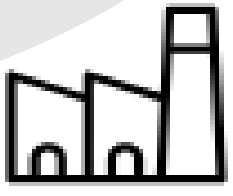
-for beginners

Vignesh Moorthi

Mar - 2025

What is Kafka?

Apache Kafka is an open-source distributed event streaming platform used by thousands of companies for high-performance data pipelines, streaming analytics, data integration, and mission-critical applications.



10 / 10
Manufacturing

7 / 10
Banks

10 / 10
Insurance

8 / 10
Telecom

Kafka Architecture

- Kafka is used as a pub-sub messaging middleware, there are **3 important components**.

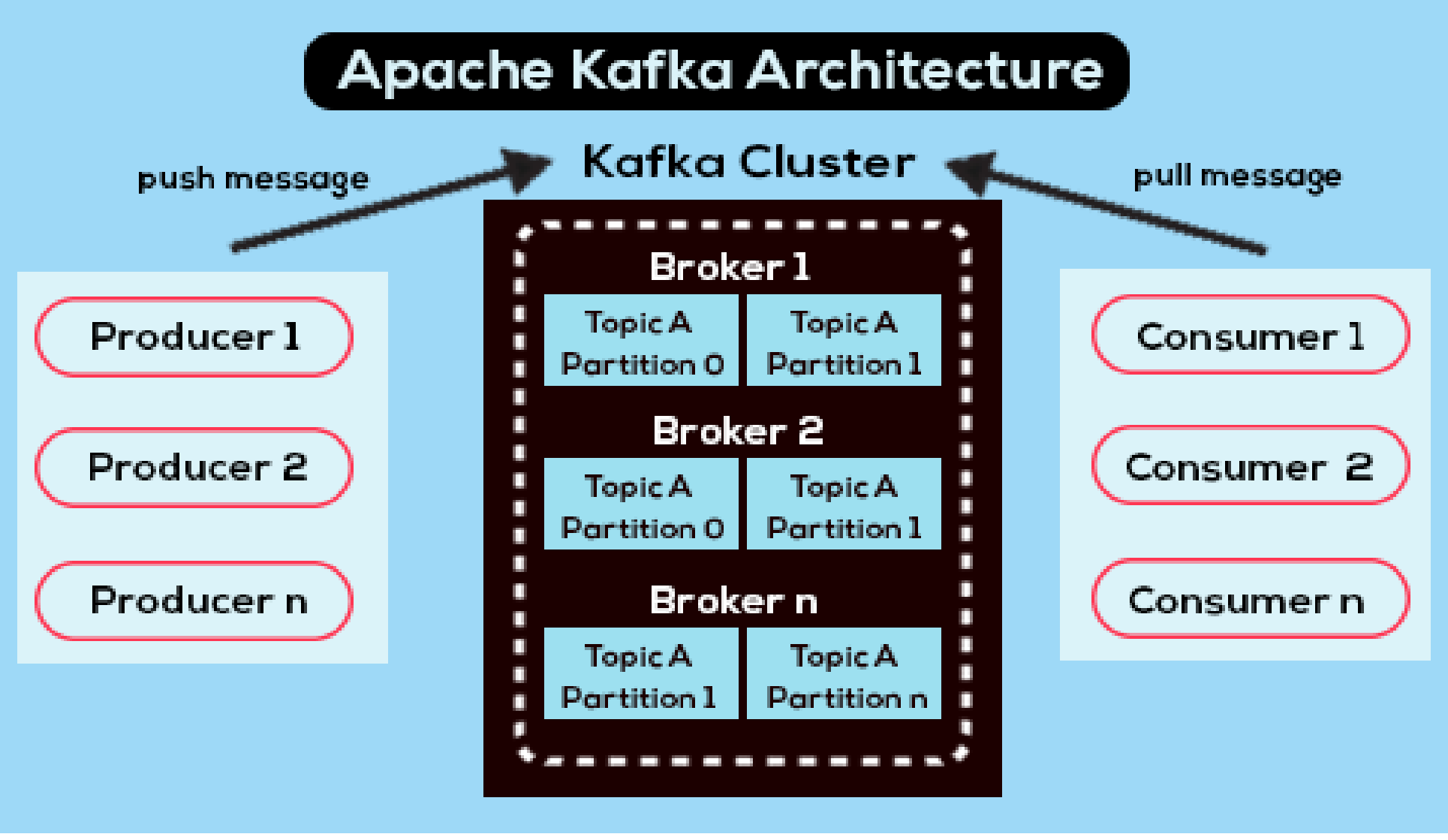
  - Producer
  - Broker
  - Consumer
  - Topics

- **Producer :** The producer is the message sender, and the consumer is the message receiver.

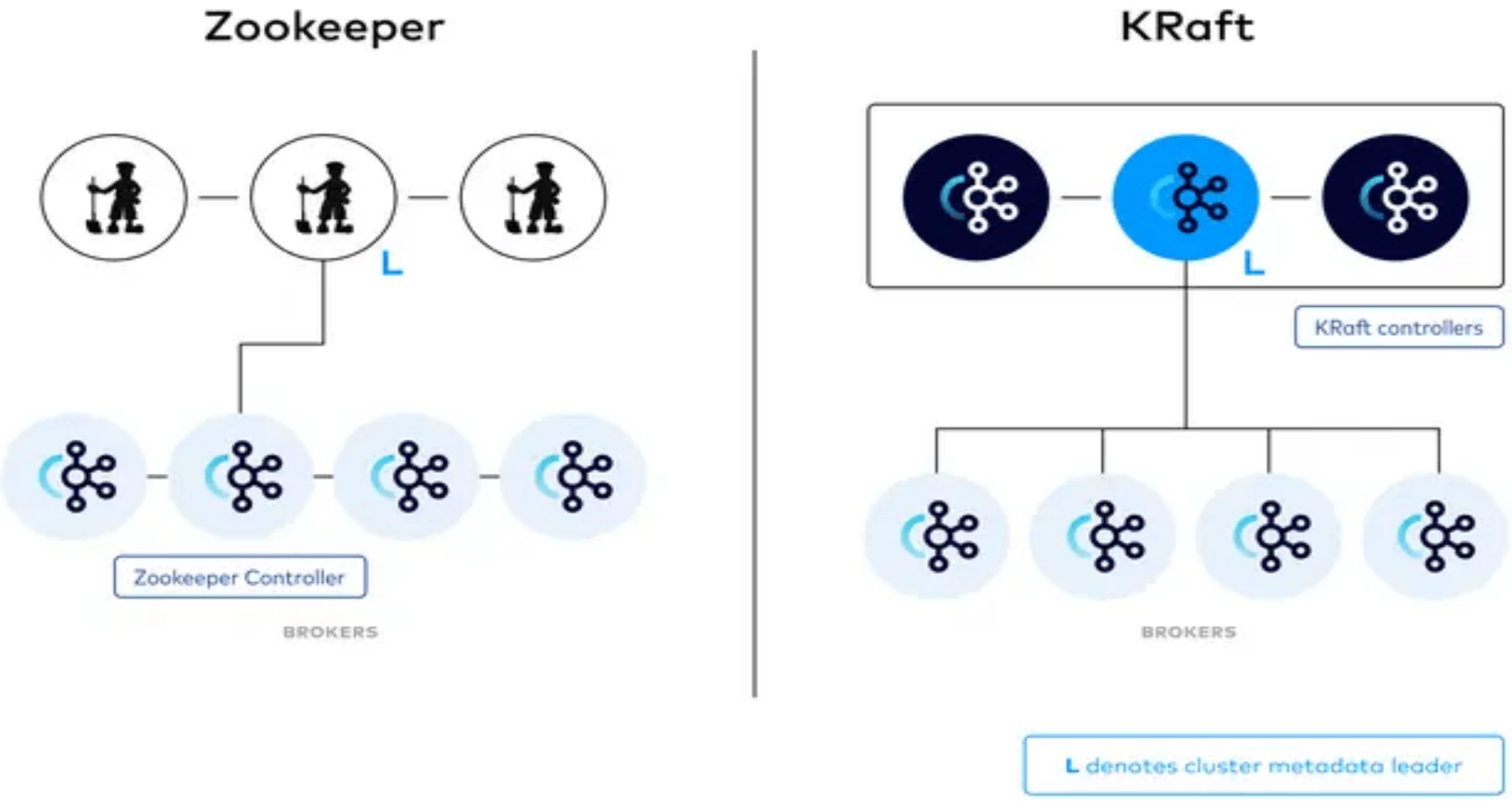
Broker : The broker is usually deployed in a cluster mode, which handles incoming messages and writes them to the broker partitions.

Consumers : It is a client application that subscribes to and reads data (events) from specific topics or partitions, processing the data in real-time

Topics : It acts as a named logical channel where producers publish messages and consumers read them, effectively organizing data streams and acting as a central hub for real-time event processing.


Apache Kafka Architecture
push message
Kafka Cluster
pull message
Producer 1
Producer 2
Producer n
Broker 1
Topic A Partition 0
Topic A Partition 1
Broker 2
Topic A Partition 0
Topic A Partition 1
Broker n
Topic A Partition 1
Topic A Partition n
Consumer 1
Consumer 2
Consumer n

Metadata Management and Cluster Coordination



ZooKeeper :

Role: External Coordination Service

Function: ZooKeeper acts as a centralized service for maintaining configuration information, naming, providing distributed synchronization, and providing group services. In Kafka, it is primarily used for:

- Managing metadata about Kafka topics, partitions, and brokers.
- Handling leader election for partitions.
- Keeping track of the cluster state and membership.

KRaft :

Role: Internal Consensus Mechanism

Function: KRaft is designed to manage Kafka's metadata directly within the Kafka brokers, eliminating the need for an external service like ZooKeeper. It provides:

- A built-in consensus algorithm (Raft) for managing metadata.
- Improved scalability and performance by integrating metadata management into Kafka's architecture.
- Simplified deployment and operational management by removing the dependency on ZooKeeper.

Feature/Aspect	ZooKeeper	KRaft (Kafka Raft)
Architecture	External coordination service	Integrated consensus mechanism within Kafka
Metadata Management	Manages metadata separately from Kafka brokers	Manages metadata directly within Kafka brokers
Consensus Algorithm	Uses a leader-follower model	Uses the Raft consensus algorithm
Operational Complexity	Requires separate installation and management	Simplifies deployment by eliminating ZooKeeper.
Scalability	Can become a bottleneck with large clusters	Designed for better scalability with Kafka's architecture
Leader Election	Centralized leader election process	Faster decentralized leader election
Fault Tolerance	Can lead to stale metadata during transitions	Provides strong consistency and fault tolerance
Performance	Performance can degrade with high request rates	Optimized for high throughput and low latency
Configuration Management	Stores configurations for topics and brokers	Configuration is managed within Kafka topics
Future Development	Planned deprecation in future Kafka releases	Represents the future direction of Kafka

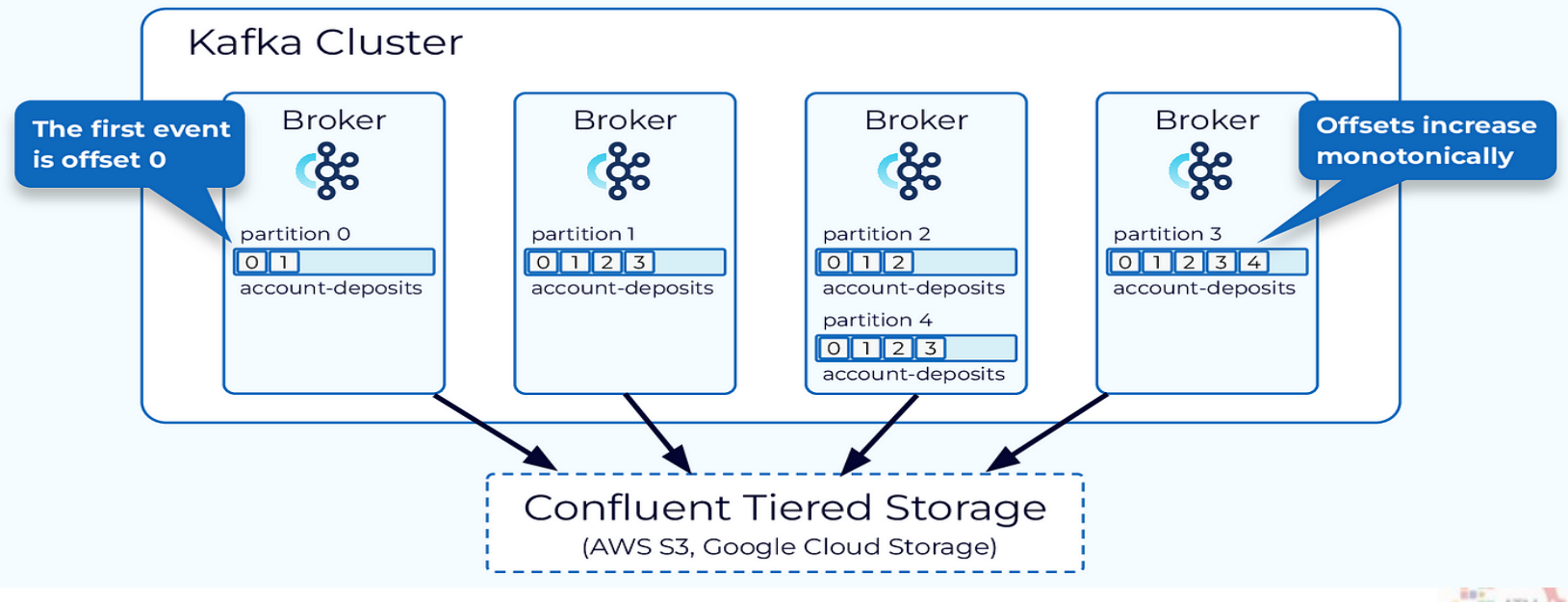
Layer of Kafka

- Compute Layer
- Storage Layer

Compute Layer :

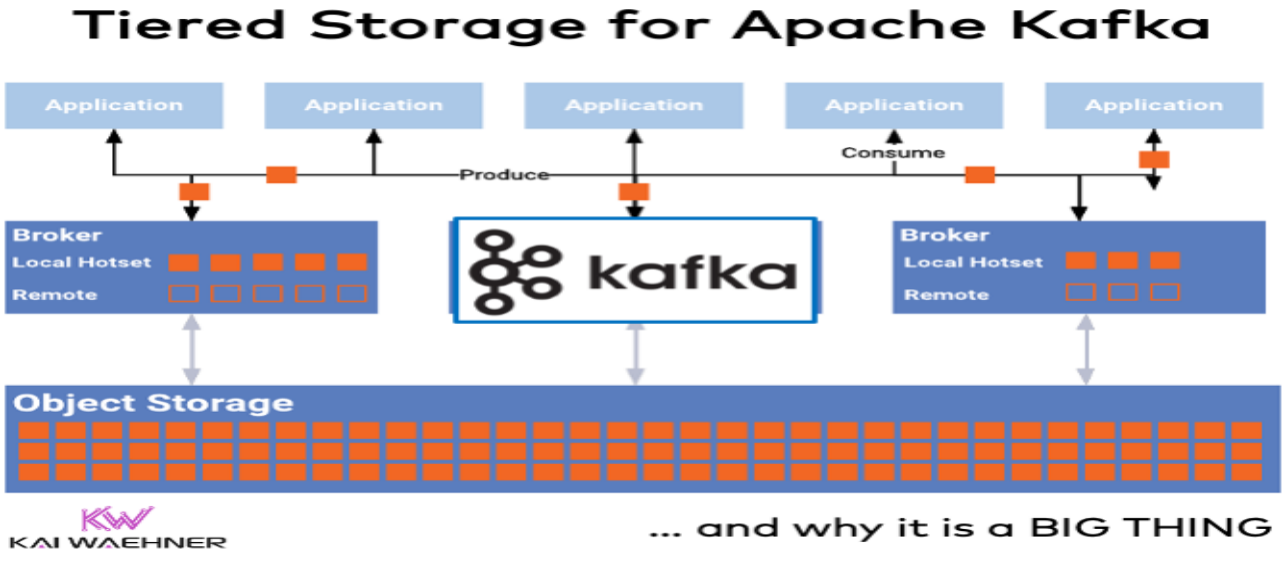
- The compute layer, or the processing layer, allows various applications to communicate with Kafka brokers via APIs.
- The **compute layer** in Kafka refers to the processing capabilities that allow for real-time data processing and analytics. While Kafka itself is primarily a messaging system, it provides several features and integrations that enable stream processing.

- **Kafka Streams -** is a powerful stream processing library that allows developers to build applications that process data in real-time. It enables operations such as filtering, aggregating, joining, and transforming data streams.
- **KSQL (Kafka SQL) -** is a SQL-like query language for stream processing in Kafka. It allows users to perform real-time analytics on data streams using familiar SQL syntax, making it accessible to a broader audience.

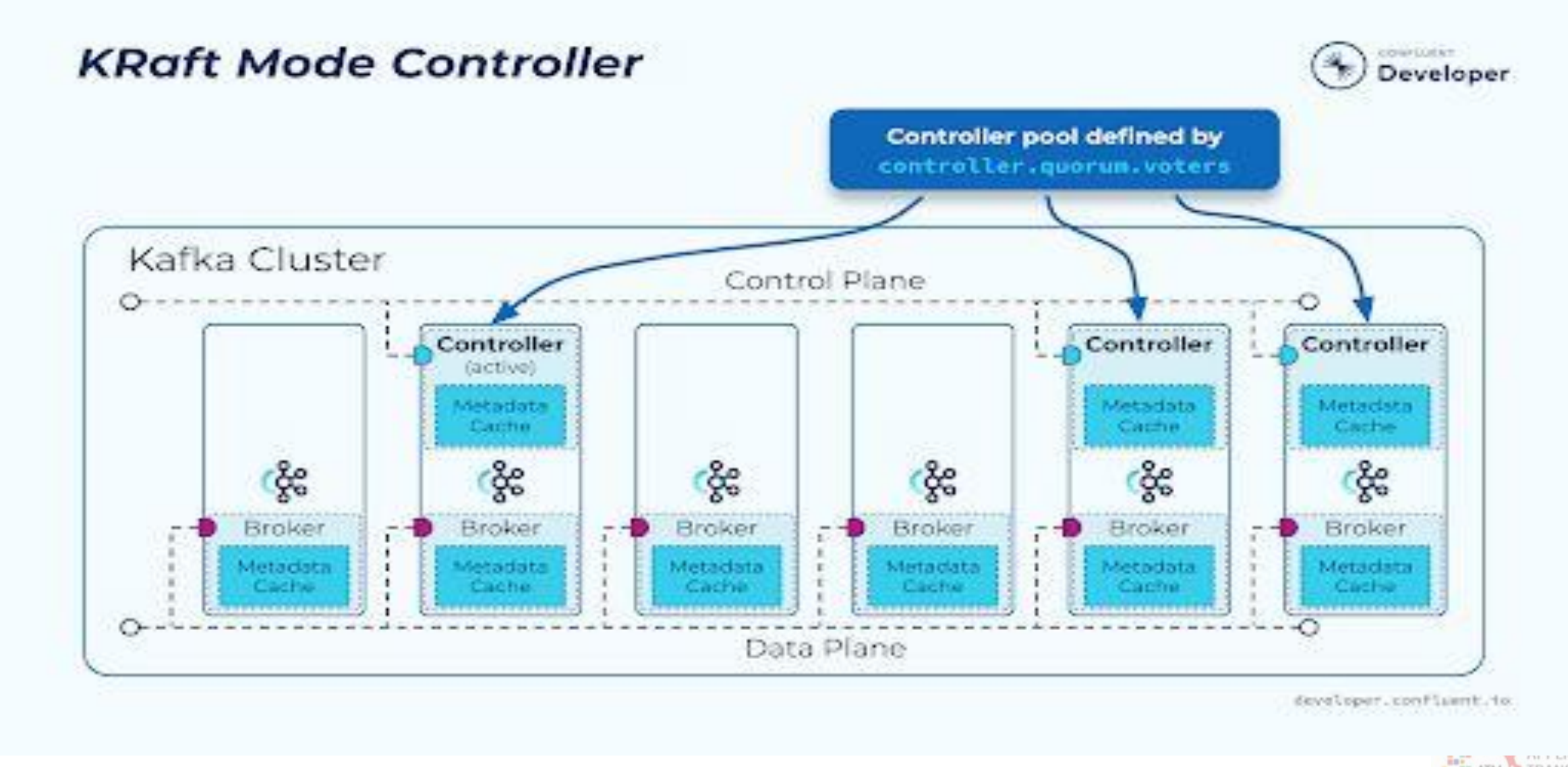


Storage Layer :

- This layer is composed of Kafka brokers. Kafka brokers run on a cluster of servers. The data is stored in partitions within different topics. A topic is like a database table, and the partitions in a topic can be distributed across the cluster nodes. Within a partition, events are strictly ordered by their offsets.



Mode Controller :

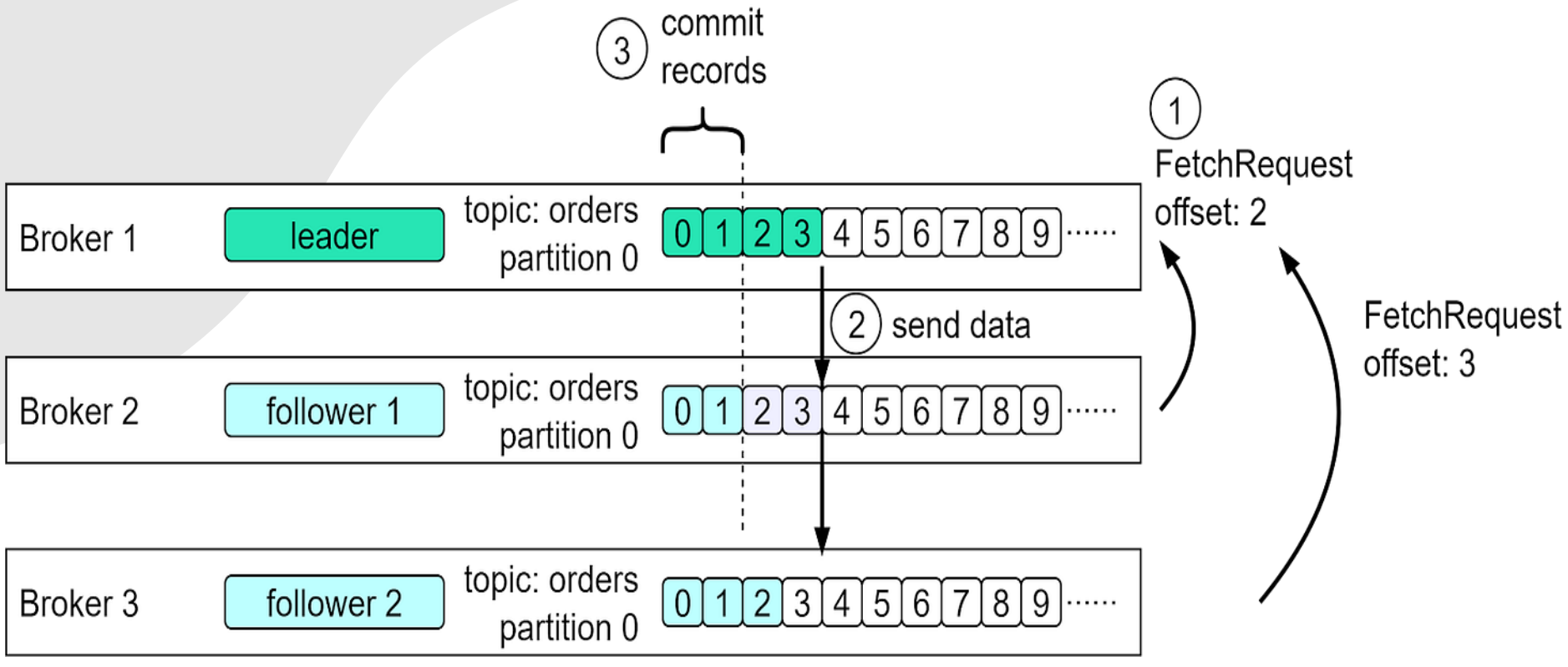


Control Plane Mode :

- The control plane **manages the metadata of the cluster, including tasks like broker management, partition assignments, and leader election**, while the data plane handles the actual data flow and replication

- The control plane in Kafka refers to the components and processes responsible for managing the configuration, orchestration, and coordination of Kafka brokers, topics, partitions, and consumer groups

Data Plane Mode :



The data plane handles the data replication. The diagram below shows an example. Partition 0 in the topic “orders” has 3 replicas on the 3 brokers. The partition on Broker 1 is the leader, where the current data offset is at 4; the partitions on Broker 2 and 3 are the followers where the offsets are at 2 and 3.

Step 1 - In order to catch up with the leader, Follower 1 issues a Fetch Request with offset 2, and Follower 2 issues a Fetch Request with offset 3.

Step 2 - The leader then sends the data to the two followers accordingly.

Step 3 - Since followers’ requests implicitly confirm the receipts of previously fetched records, the leader then commits the records before offset 2.

Step 7: The producer requests are stashed into a purgatory structure for replication, so the I/O thread can be freed up to pick up the next request.

Step 8: Once a request is replicated, it is removed from the purgatory. A response is generated and put into the response queue.

Steps 9 and 10: The network thread picks up the response from the response queue and sends it to the corresponding socket send buffer. Note that the network thread is bound to a certain client. Only after the response for a request is sent out, will the network thread take another request from the particular client.

Records :

Kafka uses the Record class as an abstraction of an event.

There are 4 parts in a Record :

- Timestamp
- Key
- Value
- Headers (optional)

The key is used for enforcing ordering, colocating the data that has the same key, and data retention.

Broker

HEXAWARE

We discussed brokers as the storage layer. The data is organized in topics and stored as partitions on the brokers. Now let's look at how a broker works in detail.

Step 1: The producer sends a request to the broker, which lands in the broker's socket receive buffer first.

Steps 2 and 3: One of the network threads picks up the request from the socket receive buffer and puts it into the shared request queue. The thread is bound to the particular producer client.

Step 4: Kafka's I/O thread pool picks up the request from the request queue.

Steps 5 and 6: The I/O thread validates the CRC of the data and appends it to commit log. The commit log is organized on disk in segments. There are two parts in each segment: the actual data and the index.

ATM APPLICATION TRANSFORMATION MANAGEMENT

Messaging
Metrics
Website Activity Tracking
Log Aggregation
Stream Processing
Event Sourcing
Commit Log
Apache Kafka
Use Cases

Messaging

Kafka works well as a replacement for a more traditional message broker. Message brokers are used for a variety of reasons (to decouple processing from data producers, to buffer unprocessed messages, etc).

As a messaging system. **For example**, Kafka can be used to process payments and financial transactions in **real-time, such as in stock exchanges, banks, and insurance companies**.

In our experience messaging uses are often comparatively low-throughput but may require low end-to-end latency and often depend on the strong durability guarantees Kafka provides.

Website Activity Tracking

The original use case for Kafka was to be able to rebuild a user activity tracking pipeline as a **set of real-time publish-subscribe feeds.**

This means site activity (page views, searches, or other actions users may take) is published to central topics with one topic per activity type. These feeds are available for subscription for a range of use cases including **real-time processing, real-time monitoring, and loading into Hadoop or offline data warehousing systems for offline processing and reporting.**

For example, Kafka can be used to track and monitor cars, trucks, fleets, and shipments in real-time, such as for taxi services, in logistics and the automotive industry.

Metrics

KAFKA IS OFTEN USED FOR OPERATIONAL MONITORING DATA. THIS INVOLVES AGGREGATING STATISTICS FROM DISTRIBUTED APPLICATIONS TO PRODUCE CENTRALIZED FEEDS OF OPERATIONAL DATA.

TO GATHER METRICS DATA. **FOR EXAMPLE,** KAFKA CAN BE USED TO CONTINUOUSLY CAPTURE AND ANALYZE SENSOR DATA FROM IOT DEVICES OR OTHER EQUIPMENT, SUCH AS IN FACTORIES AND WIND PARKS.

Log Aggregation

- It is the process of **collecting and consolidating log data from multiple sources into a centralized location for analysis, monitoring, and management**. This practice is essential for organizations that operate distributed systems, as it allows them to efficiently manage and analyze logs generated by various applications, services, and infrastructure components.
- This allows for lower-latency processing and easier support for multiple data sources and distributed data consumption.
- In comparison to log-centric systems like **Scribe or Flume, Kafka offers equally good performance, stronger durability guarantees due to replication, and much lower end-to-end latency.**

Stream Processing

Apache Kafka is widely used for stream processing due to its ability to handle real-time data streams efficiently. Common use cases include real-time analytics, fraud detection, data integration, and monitoring applications, allowing organizations to process and respond to data as it flows in.

Starting in 0.10.0.0, a light-weight but powerful stream processing library called Kafka Streams is available in Apache Kafka to perform such data processing as described above. Apart from Kafka Streams, alternative open source, stream processing tools include Apache Storm and Apache Samza.

For example, use Kafka to collect and react to customer interactions and orders, such as in retail, the hotel and travel industry, and mobile applications.

Event Sourcing

It is a style of application design where state changes are logged as a time-ordered sequence of records. Kafka's support for very large stored log data makes it an excellent backend for an application built in this style.

For example, Let's consider an e-commerce order management system to illustrate event sourcing.

Commit Log

Kafka can serve as a kind of external commit-log for a distributed system. The log helps replicate data between nodes and acts as a re-syncing mechanism for failed nodes to restore their data.

For example, Companies like Uber and Netflix use Kafka as a messaging backbone for inter-service communication, allowing microservices to exchange events and data in a scalable, decoupled manner.